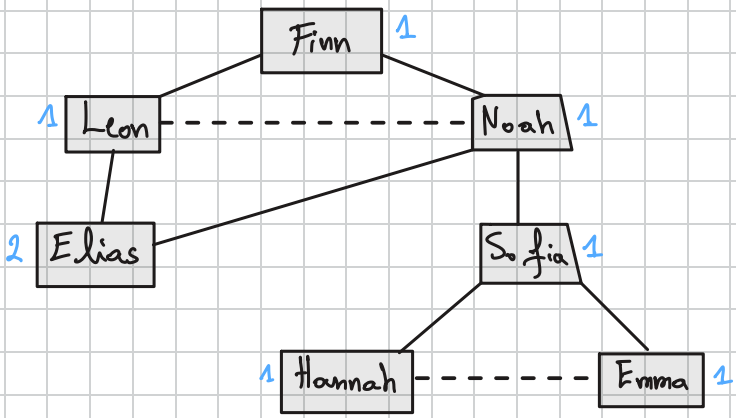
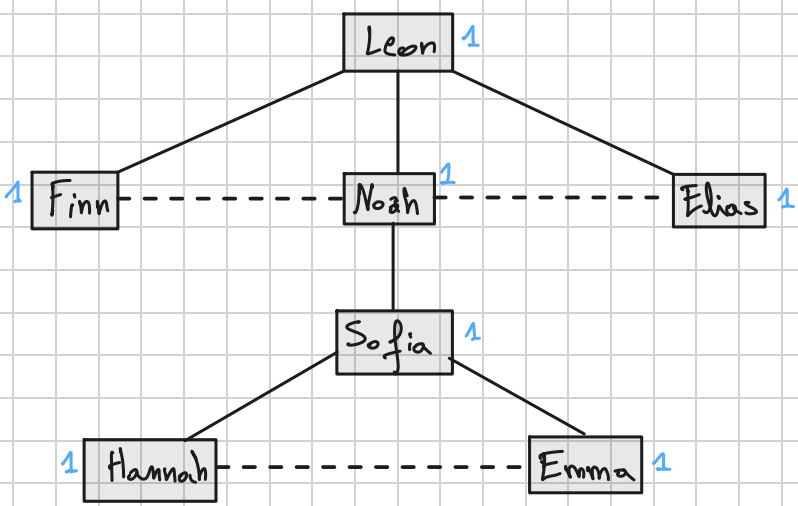
Assignment 4:

Task 1:

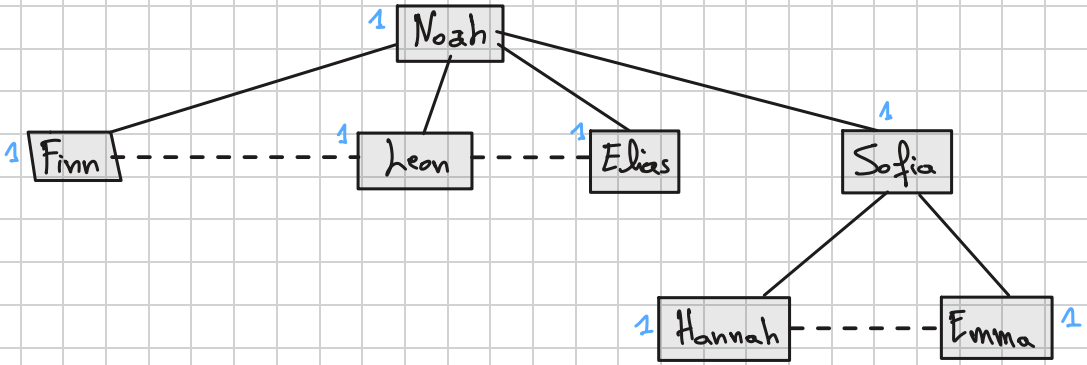
(a) Let's take the node "Finn" as the root node:



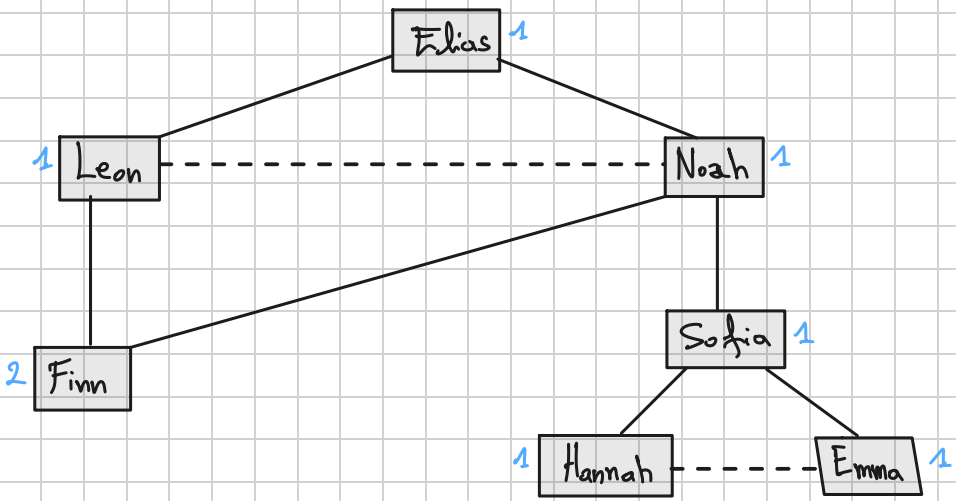
- Let's take the node "Leon" as the root node:



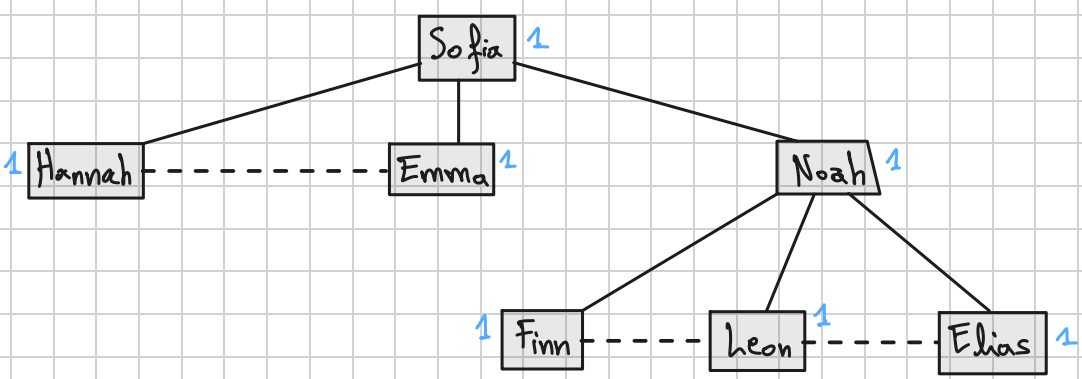
- Let's take the node "Noah" as the root node:



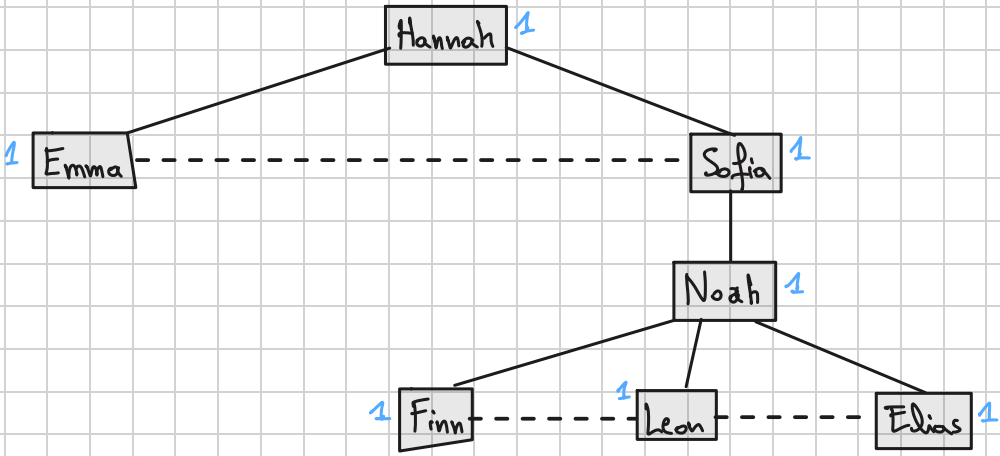
- Let's take the node "Elias" as the root node:



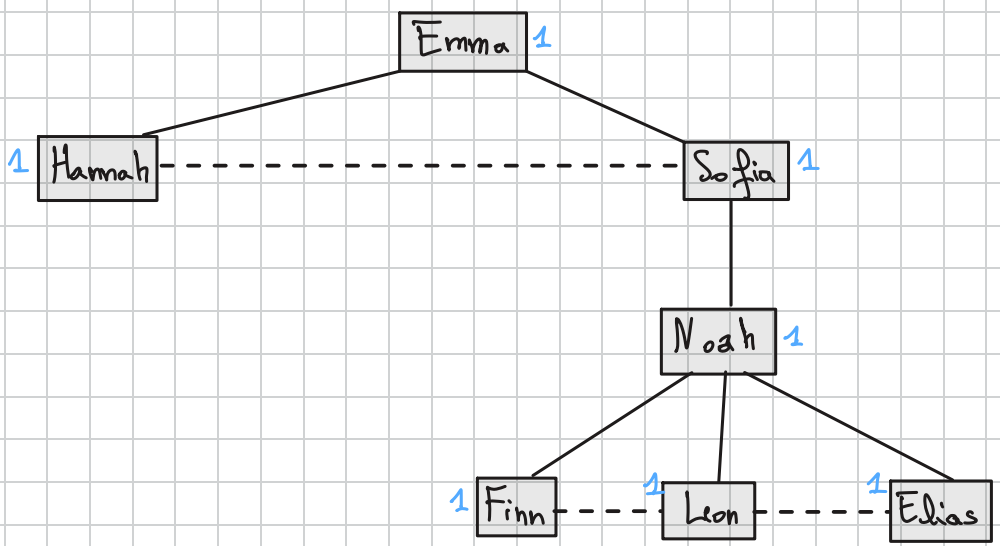
- Let's take the node "Sofia" as the root node:



- Let's take the node "Hannah" as the root node:



- Let's take the node "Emma" as the root node:

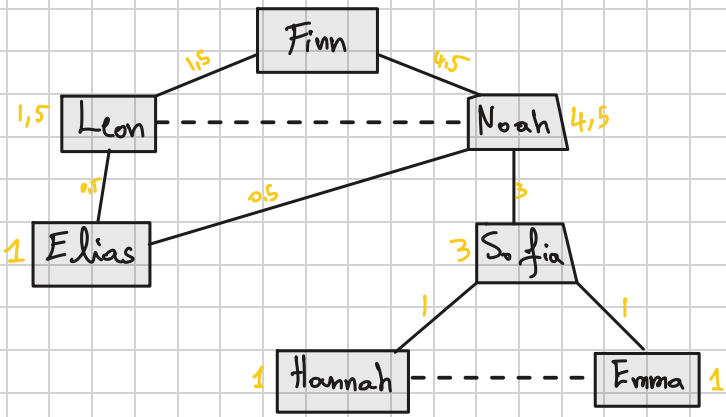


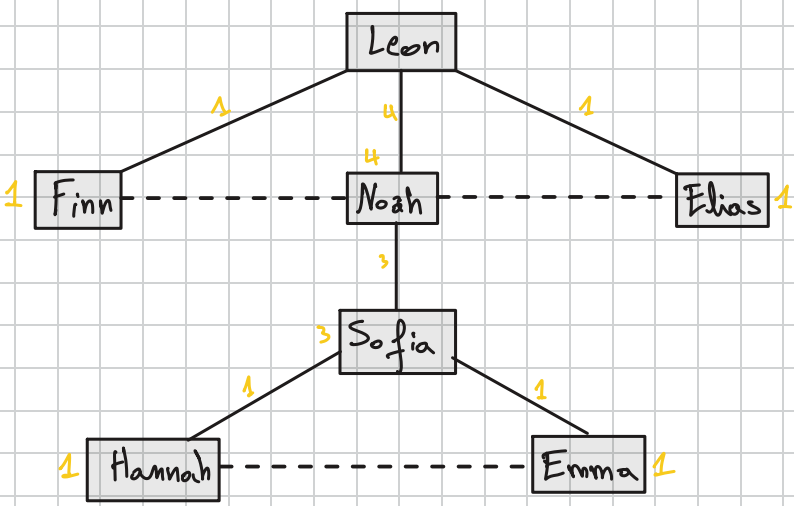
(b) Let's label each node by the number of shortest paths that reach it from the root (we will add labels above without redrawing the graphs again)

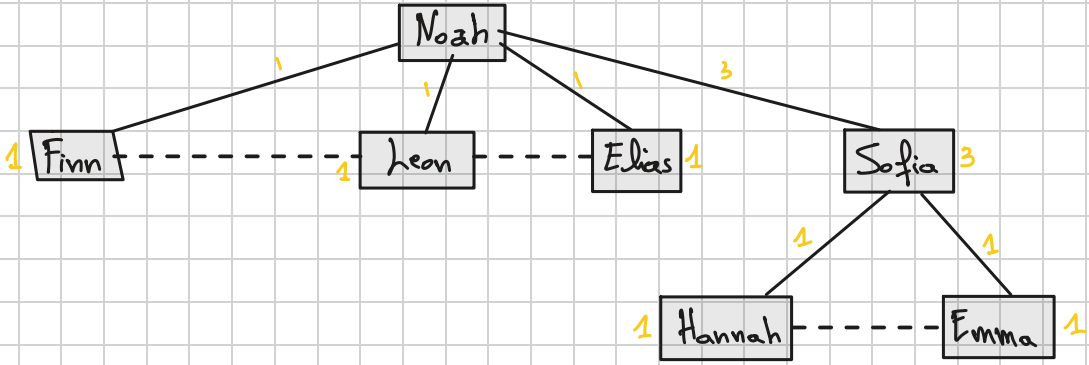
⇒ We label the root with 1, and the other nodes are labeled by

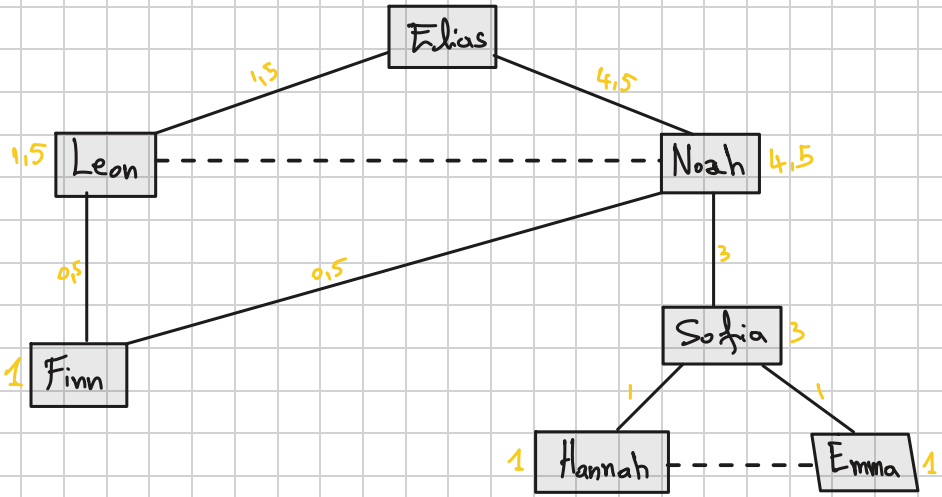
the sum of their parents. We do that to count the number of shortest paths from the root to a specific node.

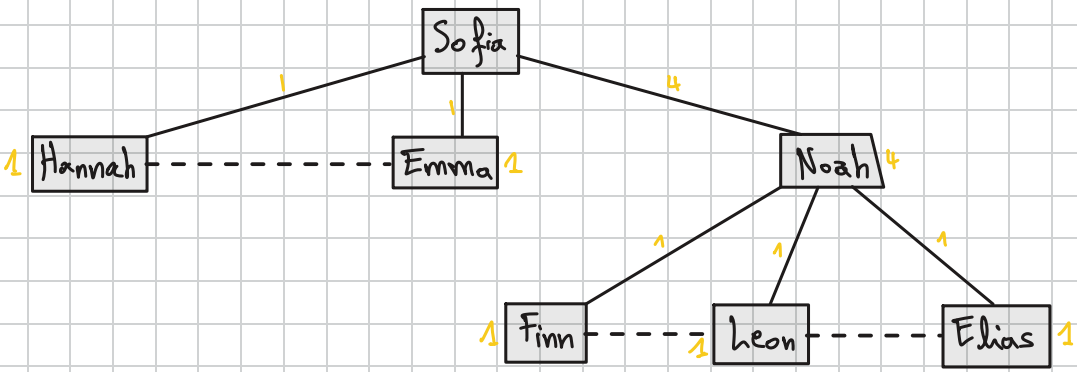
(c) Let's assign a credit value of 1 to each leaf node (we will duplicate the graphs down here to prevent messy data)

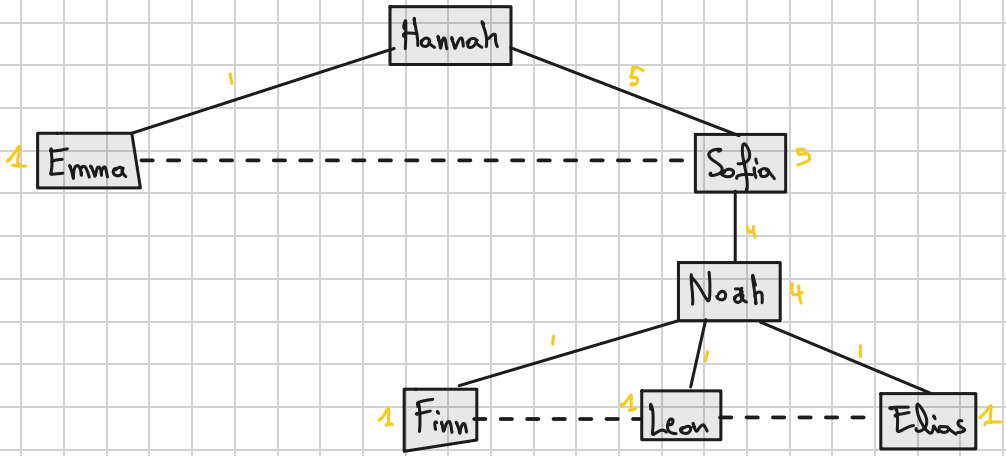


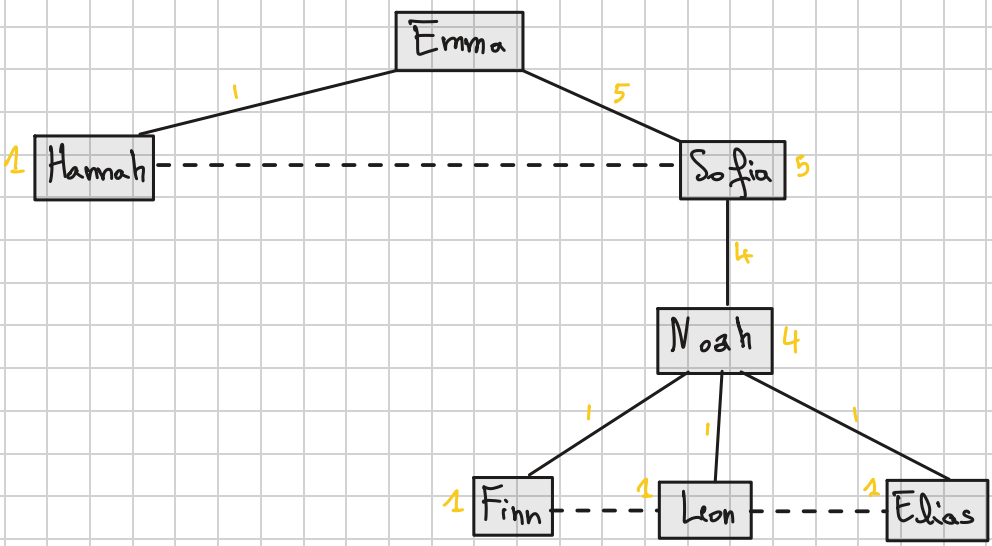



Noah
Finn
Leon
Elias
Sofia
Hannah
Emma
1
1
1
3
1
1
1
1
1
1
3
1
1
1


Elias
Leon
Noah
Finn
Sofia
Hannah
Emma
1,5
4,5
1,5
4,5
0,5
0,5
3
3
1
1
1
1
1
1


Sofia
Hannah
Emma
Noah
Finn
Leon
Elias
1
1
4
1
1
1
1
1
1
4
1
1
1



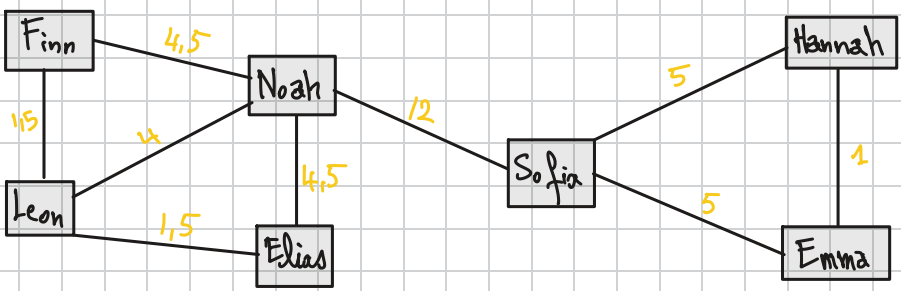


(d) Let's compute the node credit for all non-leaf nodes (we will write it on the graphs above)

(e) - Edge (Finn, Leon): $\frac{1,5+1+0+0,5+0+0+0}{2} = \frac{3}{2} = 1,5$

- Edge (Finn, Noah): $\frac{4,5+0+1+0,5+1+1+1}{2} = \frac{9}{2} = 4,5$

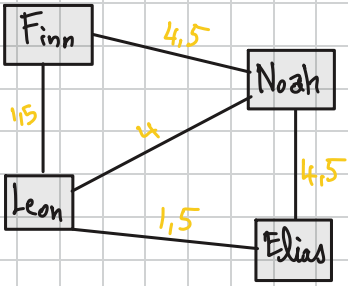
- Edge (Leon, Noah): $\frac{0+4+1+0+1+1+1}{2} = \frac{8}{2} = 4$
- Edge (Leon, Ilias): $\frac{0,5+1+0+1,5+0+0+0}{2} = \frac{3}{2} = 1,5$
- Edge (Noah, Ilias): $\frac{0,5+0+1+4,5+1+1+1}{2} = \frac{9}{2} = 4,5$
- Edge (Noah, Sofia): $\frac{3+3+3+3+4+4+4}{2} = \frac{24}{2} = 12$
- Edge (Sofia, Hannah): $\frac{1+1+1+1+1+5+0}{2} = \frac{10}{2} = 5$
- Edge (Sofia, Emma): $\frac{1+1+1+1+1+0+5}{2} = \frac{10}{2} = 5$
- Edge (Hannah, Emma): $\frac{0+0+0+0+0+1+1}{2} = \frac{2}{2} = 1$

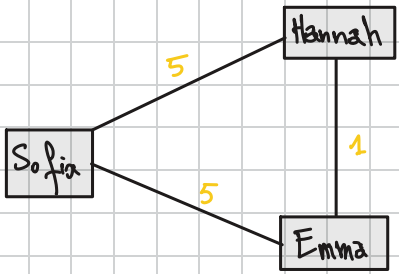


(f) Edge betweenness centrality is used to count the shortest path between every single pair of nodes, and assigns the number of times each edge was used to complete those paths. The edges that have the highest betweenness score assigned to it is the one connecting two different communities.

For this graph, the edge (Noah, Sofia) has the highest betweenness score of 12, by removing it we will split the network into

two separate communities.





after the first iteration we will have two separate communities

{ Finn, Leon, Elias, Noah } and { Sofia, Emma, Hannah }

Task 2:

(a) The betweenness centrality in a standard graph measures how much control a node has over the graph, that means how frequent that node is as the shortest path between two other nodes. For our E-commerce database, betweenness shows what are the nodes that bridge different groups of users and products.

If a product has high betweenness centrality, it means that it is a product that was bought by different types of users, it's like a bridge between them. And for a user to have a high betweenness centrality, it means that this user have bought different products from different categories, it kind of acts like a path connecting

all these distinct products.

The difference between our database and a classic social network, is that our users are not directly connected to each other. In a social network, high betweenness centrality means a person bridges two different social networks, but in our case, users are connected indirectly based on the same products they have purchased or reviewed.

(b)

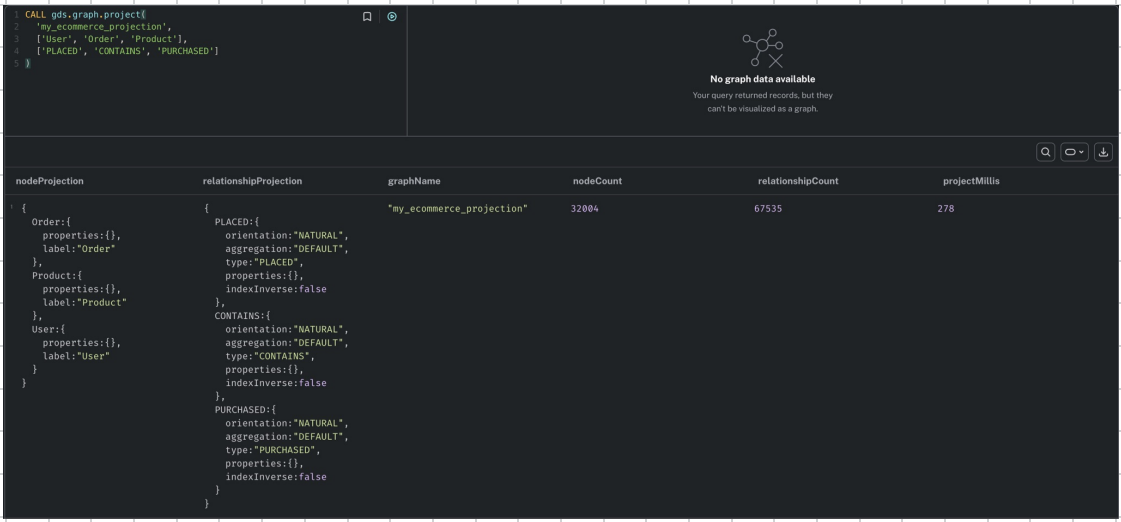
```
CALL gds.betweenness.stream(
    graphName: String,
    configuration: Map
)
YIELD nodeId: Integer, score: Float
```

- calling from graph data science library
- the exact algorithm
- execution mode (without saving)
- cypher keyword to trigger a built-in algorithm
- name of the specific portion of the database
- Dictionary where to pass extra configuration parameters
- Keyword to catch the output
- id of the node
- betweenness centrality score

This query uses the betweenness algorithm from the GDS library to calculate the betweenness centrality for each node in a specified graph, it then output the results in two variables: the node ID and its score.

(c)

```
CALL gds.graph.project(
  'my_ecommerce_projection',
  ['User', 'Order', 'Product'],
  ['PLACED', 'CONTAINS', 'PURCHASED']
)
```



```
CALL gds.graph.project(
  'my_ecommerce_projection',
  ['User', 'Order', 'Product'],
  ['PLACED', 'CONTAINS', 'PURCHASED']
)
```

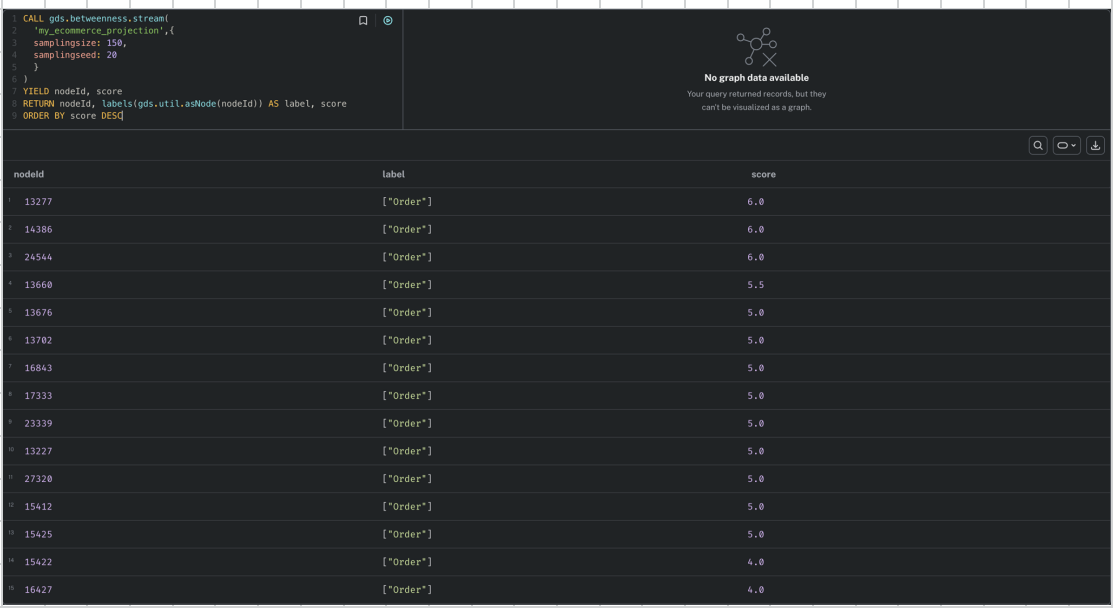
No graph data available

Your query returned records, but they can't be visualized as a graph.

nodeProjection	relationshipProjection	graphName	nodeCount	relationshipCount	projectMillis
{ Order:{ properties:{}, label:"Order" }, Product:{ properties:{}, label:"Product" }, User:{ properties:{}, label:"User" } }	{ PLACED:{ orientation:"NATURAL", aggregation:"DEFAULT", type:"PLACED", properties:{}, indexInverse:false }, CONTAINS:{ orientation:"NATURAL", aggregation:"DEFAULT", type:"CONTAINS", properties:{}, indexInverse:false }, PURCHASED:{ orientation:"NATURAL", aggregation:"DEFAULT", type:"PURCHASED", properties:{}, indexInverse:false } }	"my_ecommerce_projection"	32004	67535	278

We selected the "User", "Product" and "Order" nodes with "PLACED", "CONTAINS" and "PURCHASED" edges because we think they are the most critical part in this e-commerce database, that they show the actual commercial behavior of the graph.

```
CALL gds.betweenness.stream(
  'my_ecommerce_projection',{
  samplingsize: 150,
  samplingseed: 20
  }
)
YIELD nodeId, score
RETURN nodeId, labels(gds.util.asNode(nodeId)) AS label, score
ORDER BY score DESC
```



	nodeId	label	score
1	13277	["Order"]	6.0
2	14386	["Order"]	6.0
3	24544	["Order"]	6.0
4	13660	["Order"]	5.5
5	13676	["Order"]	5.0
6	13702	["Order"]	5.0
7	16843	["Order"]	5.0
8	17333	["Order"]	5.0
9	23339	["Order"]	5.0
10	13227	["Order"]	5.0
11	27320	["Order"]	5.0
12	15412	["Order"]	5.0
13	15425	["Order"]	5.0
14	15422	["Order"]	4.0
15	16427	["Order"]	4.0

We chose samplingSize as non default configuration parameter because a database can have billion of nodes and edges (and in our chosen graph projection, we have exactly 32004 nodes and 67535 edges). So, for calculating the betweenness score of each one will take a

massive amount of time and memory. Thus, by using samplingSize we use a limited random number of nodes, but we lose a bit of accuracy, and we also have to set the samplingSeed to a specific number to force the random number generator to start from the same point every time, ensuring we have the same result in each execution.

➡ The output table shows the highest scores belong to the "Order" nodes. That is because users and products are not connected directly, but via the order, and the high score indicates a massive shopping carts that connects a single user to a wide variety of products.

(d)

```
CALL gds.graph.project(
  'ecommerce_undirected_user_product',
  ['User', 'Product'],
  {
    PURCHASED: {
      orientation: 'UNDIRECTED'
    }
  }
);
```

We created first a projection with the "User" and "Product" nodes and "Purchased" edge. We made the orientation of the edge "undirected" because if we didn't, we will have all results as "1.00", because

each node can only reach the node directly next to it.

```
CALL gds.graph.project(
  'ecommerce_undirected_user_product',
  ['User', 'Product'],
  {
    PURCHASED: {
      orientation: 'UNDIRECTED'
    }
  }
);
```

Table | RAW

nodeProjection	relationshipProjection	graphName	nodeCount	relationshipCount	projectMillis

{
Product:{
properties:
{},
label:"Produc
t"
},
User:{
properties:
{},
label:"User"
}
}
	{
	PURCHASED:{
	orientation:"U
	NDIRECTED",
	aggregation:"D
	EFAULT",
	type:"PURCHASE
	D",
	properties:{},
	indexInverse:f
	alse
	}
	}
		"ecommerce_undire
		cted_user_produc
		t"
			12002	8028	176

Started streaming 1 record after 57 ms and completed after 318 ms.

CALL
gds.closeness.stream('ecommerce_undirected_user_product')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS entity, score
RETURN coalesce(entity.name, entity.product_name) AS EntityName, score
ORDER BY score DESC

Then, we call the closeness algorithm of the GDS library with the projection we created before, we take the nodeId and the score. We use the "coalesce" function to return the name field of the node ("name" for user, and "product_name" for product).

```
CALL gds.closeness.stream('ecommerce_undirected_user_product')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS entity, score
RETURN coalesce(entity.name, entity.product_name) AS EntityName, score
ORDER BY score DESC
```

Table RAW

	EntityName	score
1	"David Walls"	1.0
2	"Rodney Owens"	1.0
3	"Scott Thornton"	1.0
4	"Cody Simmons"	1.0
5	"William Ford"	1.0
6	"Susan Hayes"	1.0
7	"Rhonda Mendez"	1.0
8	"Anne Brown"	1.0
9	"Jennifer Freeman"	1.0
10	"Hannah Miller"	1.0

Fetch limit hit at 5,000 records. Started streaming after 62 ms and completed after 184 ms.

- Difference between betweenness and closeness centrality:

Closeness centrality measures the average shortest path distance from a node to all other reachable nodes in the network. A node with high closeness score means it is in the center of the network. While betweenness centrality is about measuring how often a node is between two other nodes, making it a bridge between two communities.

- A high closeness centrality for a User means a user that buys a lot of products. And a high closeness for a Product means a product is bought by many users.